

# DigitalPulse

統合型心電図解析・患者管理システム

## 詳細設計書

文書番号：DP-DD-001

版数：1.0

作成日：2026年5月5日

作成者：辰田

上位文書：DP-BD-001 基本設計書

## 改訂履歴

| 版数  | 改訂日       | 改訂者 | 改訂内容 |
|-----|-----------|-----|------|
| 1.0 | 2026年5月5日 | 辰田  | 初版作成 |

# 目次

|                             |    |
|-----------------------------|----|
| 改訂履歴                        | 2  |
| 目次                          | 3  |
| 1. はじめに                     | 4  |
| 1.1 本書の目的                   | 4  |
| 1.2 本書の対象読者                 | 4  |
| 2. バックエンドAPI詳細設計 (ecg-api)  | 5  |
| 2.1 プロジェクト構成                | 5  |
| 2.2 エンティティ詳細設計              | 6  |
| 2.2.1 EcgRecord エンティティ      | 6  |
| 2.2.2 Patient エンティティ        | 6  |
| 2.2.3 User エンティティ           | 7  |
| 2.3 リポジトリ詳細設計               | 8  |
| 2.3.1 EcgRecordRepository   | 8  |
| 2.3.2 PatientRepository     | 8  |
| 2.3.3 UserRepository        | 8  |
| 2.4 サービス詳細設計                | 9  |
| 2.4.1 AiInferenceService    | 9  |
| 2.4.2 EcgImportService      | 9  |
| 2.5 コントローラ詳細設計              | 11 |
| 2.5.1 AuthController        | 11 |
| 2.5.2 EcgRecordController   | 11 |
| 2.5.3 EcgController         | 12 |
| 2.5.4 PatientController     | 12 |
| 2.6 DTO詳細設計                 | 13 |
| 2.6.1 EcgPredictionRequest  | 13 |
| 2.6.2 EcgPredictionResponse | 13 |
| 2.7 設定詳細設計                  | 13 |
| 2.7.1 SecurityConfig        | 13 |
| 2.7.2 WebConfig             | 13 |
| 3. AIサービス詳細設計 (ecg-ai)      | 14 |
| 3.1 サービス概要                  | 14 |
| 3.2 CNNモデルアーキテクチャ           | 14 |
| 3.3 エンドポイント詳細設計             | 15 |
| 3.3.1 POST /api/predict     | 15 |
| 3.3.2 GET /health           | 15 |
| 3.4 AIプロンプト設計               | 16 |
| 3.4.1 システムプロンプト             | 16 |
| 3.4.2 ユーザープロンプト(正常時)        | 16 |
| 3.4.3 ユーザープロンプト(異常時)        | 16 |
| 3.5 Dockerコンテナ設計            | 16 |

|                                      |           |
|--------------------------------------|-----------|
| <b>4. フロントエンド詳細設計 (ecg-frontend)</b> | <b>17</b> |
| 4.1 プロジェクト構成                         | 17        |
| 4.2 ページ詳細設計                          | 17        |
| 4.2.1 ダッシュボードページ (/)                 | 17        |
| 4.2.2 心電図詳細ページ (/records/[id])       | 18        |
| 4.2.3 患者一覧ページ (/patients)            | 18        |
| 4.3 認証フロー詳細設計                        | 18        |
| 4.3.1 AppLayoutコンポーネント               | 18        |
| 4.3.2 NextAuth CredentialsProvider   | 19        |
| 4.4 コンポーネント型定義                       | 19        |
| 4.4.1 EcgRecord型                     | 19        |
| 4.4.2 Patient型                       | 19        |
| <b>5. データベース詳細設計</b>                 | <b>20</b> |
| 5.1 DDL定義                            | 20        |
| 5.1.1 usersテーブル                      | 20        |
| 5.1.2 patientsテーブル                   | 20        |
| 5.1.3 ecg_recordsテーブル                | 20        |
| 5.2 インデックス設計                         | 21        |
| 5.3 スキーマ移行ポリシー                       | 21        |
| <b>6. API入出力仕様詳細</b>                 | <b>22</b> |
| 6.1 POST /api/auth/login             | 22        |
| リクエスト仕様                              | 22        |
| レスポンス仕様 (成功時)                        | 22        |
| レスポンス仕様 (エラー時)                       | 22        |
| 6.2 POST /api/ecg/import             | 22        |
| リクエスト仕様                              | 22        |
| レスポンス仕様                              | 22        |
| 6.3 POST /api/predict (AIサービス)       | 23        |
| リクエスト仕様                              | 23        |
| レスポンス仕様 (成功時)                        | 23        |
| エラー仕様                                | 23        |
| <b>7. エラーハンドリング設計</b>                | <b>24</b> |
| 7.1 バックエンドAPIのエラーハンドリング方針            | 24        |
| 7.2 AIサービスのエラーハンドリング方針               | 24        |
| 7.3 フロントエンドのエラーハンドリング方針              | 24        |
| <b>8. セキュリティ詳細設計</b>                 | <b>25</b> |
| 8.1 認証詳細                             | 25        |
| 8.1.1 現行実装                           | 25        |
| 8.1.2 改善計画 (bcrypt移行)                | 25        |
| 8.2 CORS設定詳細                         | 25        |
| 8.3 環境変数管理                           | 25        |
| <b>9. テスト設計</b>                      | <b>26</b> |
| 9.1 単体テスト方針                          | 26        |
| 9.2 主要テストケース                         | 26        |

|                               |           |
|-------------------------------|-----------|
| <b>10. デプロイ手順</b>             | <b>27</b> |
| 10.1 バックエンドAPI(Render)        | 27        |
| 10.2 AIサービス(Google Cloud Run) | 27        |
| 10.3 フロントエンド(Vercel)          | 27        |
| <b>11. 承認欄</b>                | <b>28</b> |

# 1. はじめに

## 1.1 本書の目的

本書はDigitalPulse詳細設計書（文書番号：DP-DD-001）であり、上位文書「基本設計書（DP-BD-001）」に基づき、システムを構成する各モジュールのクラス設計・メソッド仕様・API仕様・データベーススキーマ・処理フローを詳細に定義する。実装担当者が本書を参照することにより、設計の意図を正確に理解しコーディングを行えることを目的とする。

## 1.2 本書の対象読者

- バックエンド実装担当者（Spring Boot / Java）
- フロントエンド実装担当者（Next.js / TypeScript）
- AIサービス実装担当者（FastAPI / Python）
- テスト担当者

## 2. バックエンドAPI詳細設計 (ecg-api)

### 2.1 プロジェクト構成

ecg-apiはSpring Boot 3.4 / Java 21で構成されるRESTful APIサーバーである。パッケージ構成は以下の通りである。

```
com.example.ecg_api
├── EcgApiApplication.java           // エントリーポイント
├── config/
│   ├── SecurityConfig.java         // Spring Security設定
│   └── WebConfig.java              // CORS設定
├── controller/
│   ├── AuthController.java         // 認証エンドポイント
│   ├── EcgController.java         // ECG解析エンドポイント
│   ├── EcgRecordController.java    // ECGレコード参照エンドポイント
│   └── PatientController.java      // 患者管理エンドポイント
├── dto/
│   ├── EcgPredictionRequest.java   // AI推論リクエストDTO
│   └── EcgPredictionResponse.java  // AI推論レスポンスDTO
├── entity/
│   ├── EcgRecord.java             // 心電図レコードエンティティ
│   ├── Patient.java               // 患者エンティティ
│   └── User.java                   // ユーザーエンティティ
├── repository/
│   ├── EcgRecordRepository.java    // ECGレコードリポジトリ
│   ├── PatientRepository.java      // 患者リポジトリ
│   └── UserRepository.java         // ユーザーリポジトリ
└── service/
    ├── AiInferenceService.java     // AIサービス呼び出し
    └── EcgImportService.java       // CSV取込・解析
```

## 2.2 エンティティ詳細設計

### 2.2.1 EcgRecord エンティティ

テーブル名：ecg\_records。心電図の1レコードを表すエンティティクラスである。

| フィールド名        | Javaの型        | DBカラム名         | 説明・アノテーション                         |
|---------------|---------------|----------------|------------------------------------|
| id            | Integer       | id             | @Id @GeneratedValue(IDENTITY) 主キー  |
| patient       | Patient       | patient_id     | @ManyToOne @JoinColumn 患者への外部キー参照  |
| recordedAt    | LocalDateTime | recorded_at    | @Column インポート日時                    |
| heartRate     | Integer       | heart_rate     | @Column 心拍数（拡張用）                   |
| isAnomaly     | Boolean       | is_anomaly     | @Column AI判定による異常フラグ               |
| waveformData  | String        | waveform_data  | @Column(json) 187点波形データ（JSON配列文字列） |
| doctorComment | String        | doctor_comment | @Column(TEXT) AI所見＋医師コメント          |
| diagnosisType | Integer       | diagnosis_type | @Column 診断分類コード（0～4）               |

### 2.2.2 Patient エンティティ

テーブル名：patients。患者の基本情報を管理するエンティティクラスである。

| フィールド名 | Javaの型  | DBカラム名 | 説明・アノテーション                        |
|--------|---------|--------|-----------------------------------|
| id     | Integer | id     | @Id @GeneratedValue(IDENTITY) 主キー |
| name   | String  | name   | 患者氏名                              |
| age    | Integer | age    | 年齢                                |
| gender | String  | gender | 性別（Male/Female/Other）             |



2.2.3 User エンティティ

テーブル名：users。システムへのログインユーザーを管理するエンティティクラスである。

| フィールド名      | Javaの型  | DBカラム名       | 説明・アノテーション                        |
|-------------|---------|--------------|-----------------------------------|
| id          | Integer | id           | @Id @GeneratedValue(IDENTITY) 主キー |
| username    | String  | username     | @Column(unique=true) ログインID       |
| password    | String  | password     | パスワード（bcryptハッシュ化予定）              |
| displayName | String  | display_name | 表示名                               |
| role        | String  | role         | 権限ロール（将来拡張用）                      |

## 2.3 リポジトリ詳細設計

### 2.3.1 EcgRecordRepository

インターフェース継承：JpaRepository<EcgRecord, Integer>

| メソッド名                               | 戻り値                 | 説明                                           |
|-------------------------------------|---------------------|----------------------------------------------|
| findAllOrderByIdDesc()              | List<EcgRecord>     | 全レコードをID降順で取得する                              |
| findTop100OrderByIdAsc()            | List<EcgRecord>     | ID昇順で上位100件を取得する（サマリ表示用）                     |
| searchRecords(patientId, isAnomaly) | List<EcgRecord>     | @Query による動的クエリ。患者ID・異常フラグでフィルタリング（null時は全件） |
| findById(id)                        | Optional<EcgRecord> | JpaRepository標準メソッド                          |

### 2.3.2 PatientRepository

インターフェース継承：JpaRepository<Patient, Integer>

| メソッド名         | 戻り値               | 説明                             |
|---------------|-------------------|--------------------------------|
| findAll()     | List<Patient>     | JpaRepository標準メソッド。全患者を取得する   |
| findById(id)  | Optional<Patient> | JpaRepository標準メソッド。IDで患者を取得する |
| save(patient) | Patient           | JpaRepository標準メソッド。患者を保存する    |

### 2.3.3 UserRepository

インターフェース継承：JpaRepository<User, Integer>

| メソッド名                    | 戻り値            | 説明                                   |
|--------------------------|----------------|--------------------------------------|
| findByUsername(username) | Optional<User> | Spring Data JPA派生クエリ。ユーザー名でユーザーを検索する |

## 2.4 サービス詳細設計

### 2.4.1 AiInferenceService

役割：FastAPIベースのAIサービスへのHTTPリクエスト送信とレスポンス受信を担う。

| メソッド名             | 引数           | 戻り値                   | 説明                                                               |
|-------------------|--------------|-----------------------|------------------------------------------------------------------|
| predict(waveform) | List<Double> | EcgPredictionResponse | AIサービスのPOST /api/predictを呼び出し、推論結果を返す。失敗時はRuntimeExceptionをスローする |

処理フロー：

- 1. EcgPredictionRequest (waveformリスト) を生成する
- 2. RestTemplateを使用してaiApiUrl + "/api/predict" へPOSTリクエストを送信する
- 3. レスポンスをEcgPredictionResponseにデシリアライズして返す
- 4. 例外発生時はRuntimeException (「AI解析の呼び出しに失敗しました」) をスローする

### 2.4.2 EcgImportService

役割：CSVファイルの解析・AI推論の呼び出し・DBへの保存を担う。Async処理による既存レコードへの一括AI適用も提供する。

| メソッド名                      | 引数                     | 処理概要                                          |
|----------------------------|------------------------|-----------------------------------------------|
| importCsv(file, patientId) | MultipartFile, Integer | CSVの各行から187点の波形を抽出し、AI推論を実行してecg_recordsに保存する |
| processExistingRecords()   | なし (@Async)            | doctor_commentが空のレコード最大50件にバックグラウンドでAI推論を適用する |

importCsv 処理フロー詳細：

- 1. patientIdが指定されている場合、PatientRepositoryで患者を検索する
- 2. UTF-8でCSVファイルを読み込む (BufferedReader + CSVParser)
- 3. 各CSVレコードの列0～186 (187列) をDouble値としてwaveformリストに追加する
- 4. AiInferenceService.predict(waveform)を呼び出してAI推論結果を取得する
- 5. EcgRecordを生成し、患者・波形データ・isAnomaly・diagnosisType・doctorComment・recordedAt (現在時刻) を設定する
- 6. doctorCommentには「【AI判定: {クラス名} (信頼度: {信頼度})】 \n{所見テキスト}」の形式で設定する
- 7. EcgRecordRepositoryに保存する

processExistingRecords 処理フロー詳細：

- 1. @Asyncアノテーションにより呼び出し元スレッドと非同期に実行する
- 2. @TransactionalアノテーションによりDBトランザクションを管理する
- 3. doctor\_commentが空のレコードをID昇順で最大50件取得する
- 4. 各レコードのwaveformDataをJSONデシリアライズしてList<Double>に変換する
- 5. AI推論を実行してisAnomaly・diagnosisType・doctorCommentを更新する
- 6. エラー発生時はエラーログを出力してそのレコードをスキップし、処理を継続する

## 2.5 コントローラ詳細設計

### 2.5.1 AuthController

ベースURL : /api/auth

| メソッド  | HTTP | パス              | リクエスト                            | レスポンス仕様                                                                  |
|-------|------|-----------------|----------------------------------|--------------------------------------------------------------------------|
| login | POST | /api/auth/login | Body:<br>{username,<br>password} | 200: {id, username}<br>/ 400: missing /<br>401: not found or<br>mismatch |

login メソッド処理フロー :

- 1. リクエストBodyからusernameとpasswordを取得する
- 2. いずれかがnullの場合、400 Bad Requestを返す
- 3. UserRepository.findByUsername(username)でユーザーを検索する
- 4. 検索結果が空の場合、401 Unauthorized (user not found) を返す
- 5. パスワードを平文比較し、不一致の場合401 Unauthorized (password mismatch) を返す
- 6. 認証成功時、{id, username}のMapを200 OKで返す

### 2.5.2 EcgRecordController

ベースURL : /api/ecg

| メソッド             | HTTP | パス                    | パラメータ                                 | 処理                               |
|------------------|------|-----------------------|---------------------------------------|----------------------------------|
| getAllEcgRecords | GET  | /api/ecg/all          | なし                                    | ID降順で全レコードをJSON配列として返す           |
| getEcgSummary    | GET  | /api/ecg/summary      | なし                                    | ID昇順で上位100件を返す                   |
| getEcgRecordById | GET  | /api/ecg/{id}         | PathVariable:<br>id                   | IDで検索した1件を返す。<br>存在しない場合はnullを返す |
| search           | GET  | /api/ecg/search       | patientId?,<br>isAnomaly?             | 患者ID・異常フラグで絞り込んで返す。パラメータなしは全件    |
| updateComment    | PUT  | /api/ecg/{id}/comment | PathVariable:<br>id, Body:<br>comment | doctor_commentを更新して保存する          |

### 2.5.3 EcgController

ベースURL : /api/ecg

| メソッド       | HTTP | パス               | パラメータ                         | 処理                                                               |
|------------|------|------------------|-------------------------------|------------------------------------------------------------------|
| test       | GET  | /api/ecg/test    | なし                            | デプロイ確認用。固定文字列を返す                                                 |
| analyzeEcg | POST | /api/ecg/analyze | Body: EcgPredictionRequest    | AiInferenceService.predictを呼び出してEcgPredictionResponseを返す         |
| importCsv  | POST | /api/ecg/import  | file (Multipart) , patientId? | EcgImportService.importCsvを呼び出す                                  |
| syncAi     | POST | /api/ecg/sync-ai | なし                            | EcgImportService.processExistingRecordsを非同期で起動し、202Acceptedを即時返す |

### 2.5.4 PatientController

ベースURL : /api/patients

| メソッド           | HTTP | パス                 | パラメータ              | 処理                    |
|----------------|------|--------------------|--------------------|-----------------------|
| getAllPatients | GET  | /api/patients      | なし                 | 全患者リストをJSONで返す        |
| createPatient  | POST | /api/patients      | Body: Patient JSON | Patient を保存して200OKで返す |
| getPatient     | GET  | /api/patients/{id} | PathVariable: id   | IDで患者を取得。存在しない場合404   |

## 2.6 DTO詳細設計

### 2.6.1 EcgPredictionRequest

AIサービスへのリクエストDTO。フィールド：waveform (List<Double>) - 187点の心電図波形データ。

### 2.6.2 EcgPredictionResponse

AIサービスからのレスポンスDTO。フィールドは以下の通りである。

| フィールド名          | 型       | 説明                                                   |
|-----------------|---------|------------------------------------------------------|
| predictionCode  | Integer | 分類コード (0:Normal, 1:SVEB, 2:VEB, 3:Fusion, 4:Unknown) |
| predictionName  | String  | 分類名 (例: "Normal (正常)")                               |
| confidence      | String  | 信頼度 (パーセント形式の文字列、例: "95.43%")                        |
| isAnomaly       | Boolean | 異常フラグ (predictionCode != 0 の場合 true)                 |
| generatedReport | String  | GPT-4o-miniが生成した所見テキスト                               |

## 2.7 設定詳細設計

### 2.7.1 SecurityConfig

Spring Securityの設定クラスである。SecurityFilterChainの設定内容は以下の通りである。

- cors : WebConfig.javaのCORS設定 (Customizer.withDefaults()) を適用する
- csrf : AbstractHttpConfigurer::disableでCSRF保護を無効化する (将来的に有効化を検討)
- authorizeHttpRequests : anyRequest().permitAll()で全エンドポイントへの認証なしアクセスを許可する

### 2.7.2 WebConfig

CORSポリシーの設定クラスである。フロントエンドURL (Vercel) とローカル開発環境 (localhost:3000) からのリクエストを許可し、credentials:includeに対応するため allowCredentials(true)を設定する。

## 3. AIサービス詳細設計 (ecg-ai)

### 3.1 サービス概要

ecg-aiはFastAPI (Python) で実装されるAI推論サービスである。起動時にTensorFlow/Kerasの学習済みモデル (ecg\_model.keras) をメモリにロードし、リクエストごとに推論を実行する。推論後はOpenAI APIを呼び出して所見テキストを生成する。

### 3.2 CNNモデルアーキテクチャ

入力形式：(1, 187, 1) - バッチサイズ1、187点の時系列、1チャンネル

| No. | レイヤー種別       | パラメータ                      | 説明                         |
|-----|--------------|----------------------------|----------------------------|
| 1   | Input        | shape=(187,1)              | 入力層。187点の波形データを受け取る        |
| 2   | Conv1D       | filters=32, kernel=5, ReLU | 第1畳み込み層。局所的な波形パターンを検出する    |
| 3   | MaxPooling1D | pool_size=2                | 第1プーリング層。特徴マップをダウンサンプリングする |
| 4   | Conv1D       | filters=64, kernel=3, ReLU | 第2畳み込み層。より高次の特徴を抽出する       |
| 5   | MaxPooling1D | pool_size=2                | 第2プーリング層。特徴マップをダウンサンプリングする |
| 6   | Flatten      | -                          | 3次元テンソルを1次元に平坦化する          |
| 7   | Dense        | units=64, ReLU             | 全結合層。64ユニットの特徴ベクトルを生成する    |
| 8   | Dense (出力)   | units=5, Softmax           | 出力層。5クラスの確率分布を出力する         |

出力：5クラスの確率分布 [Normal, SVEB, VEB, Fusion, Unknown]



## 3.3 エンドポイント詳細設計

### 3.3.1 POST /api/predict

リクエスト: EcgRequest (waveform: list[float])

レスポンス (正常時) :

```
{
  "prediction_code": 0,
  "prediction_name": "Normal (正常)",
  "confidence": "98.45%",
  "is_anomaly": false,
  "generated_report": "心電図解析の結果、正常洞調律と判定されました..."
}
```

処理フロー :

- 1. waveformリストの長さが187であることを検証する。異なる場合は400エラーを返す
- 2. waveformをnp.arrayに変換し、(1, 187, 1)にreshapeする
- 3. model.predict(input\_data)を呼び出して5クラスの確率分布を得る
- 4. np.argmax()で最大確率のクラスインデックスを取得する
- 5. CLASS\_NAMESディクショナリでクラス名を取得する
- 6. 信頼度 (確率×100) を計算する
- 7. predictionCodeが0 (Normal) かどうかでis\_anomalyを決定する
- 8. is\_anomalyに応じてOpenAI APIへのプロンプトを切り替える
- 9. gpt-4o-miniで所見テキストを生成する (temperature=0.3)
- 10. 全フィールドを含むJSONレスポンスを返す

### 3.3.2 GET /health

ヘルスチェック用エンドポイント。{"status": "ok", "model\_loaded": true/false}を返す。model\_loadedがfalseの場合、モデルのロードに失敗していることを示す。

## 3.4 AIプロンプト設計

### 3.4.1 システムプロンプト

「あなたはプロの循環器内科医をサポートする優秀な医療AIアシスタントです。専門的かつ簡潔なトーンで出力してください。」

### 3.4.2 ユーザープロンプト（正常時）

「患者の心電図をAIが解析した結果、「{クラス名}」（確信度：{信頼度}）と判定されました。医師の負担を減らすため、電子カルテにそのまま記載できる簡潔な「所見」と「方針（異常なし、経過観察など）」を2～3文で出力してください。」

### 3.4.3 ユーザープロンプト（異常時）

「患者の心電図をAIが解析した結果、「{クラス名}」（確信度：{信頼度}）と判定されました。医師の負担を減らすため、この不整脈の一般的な特徴に触れつつ、電子カルテにそのまま記載できる「所見」と、推奨される「次の方針（追加検査など）」を3～4文で出力してください。」

## 3.5 Dockerコンテナ設計

ecg-aiのDockerfileは以下のように構成する。

- ベースイメージ：Python 3.11スリムイメージ
- 依存パッケージ：requirements.txtに定義（fastapi, uvicorn, tensorflow==2.15.0, numpy, python-dotenv, openai, pydantic）
- モデルファイル：ecg\_model.kerasをコンテナイメージに同梱する
- 起動コマンド：uvicorn main:app --host 0.0.0.0 --port \${PORT:-8080}

## 4. フロントエンド詳細設計 (ecg-frontend)

### 4.1 プロジェクト構成

Next.js 15のApp Routerアーキテクチャを採用する。ルーティングはファイルシステムベースで、各page.tsxがルートに対応する。

```
ecg-frontend/
├── app/
│   ├── layout.tsx           // ルートレイアウト
│   ├── page.tsx             // ダッシュボード (/)
│   ├── api/auth/[...nextauth]/ // NextAuth APIルート
│   ├── patients/
│   │   ├── page.tsx         // 患者一覧 (/patients)
│   │   └── [id]/page.tsx     // 患者詳細 (/patients/:id)
│   └── records/
│       └── [id]/page.tsx     // ECG詳細 (/records/:id)
└── components/
    ├── AppLayout.tsx        // 認証ガード・サイドバー
    └── ...                   // 共通コンポーネント
```

### 4.2 ページ詳細設計

#### 4.2.1 ダッシュボードページ (/)

主要機能：全ECGレコードの一覧表示、患者ID・異常フラグによるフィルタリング、統計カードの表示。

| State変数                      | 用途・初期値                                          |
|------------------------------|-------------------------------------------------|
| records: EcgRecord[]         | ECGレコード一覧。初期値は空配列。APIから取得後に設定する                 |
| filteredRecords: EcgRecord[] | フィルタリング後のレコード。searchIdとanomalyFilterに基づいて更新する   |
| searchId: string             | 患者IDフィルタ文字列。URLのsearchIdパラメータで初期化する             |
| anomalyFilter: string        | 異常フィルタ ("all" / "anomaly" / "normal")。初期値は"all" |
| loading: boolean             | データ読み込み中フラグ。初期値はtrue                            |

フィルタリングロジック：searchIdとanomalyFilterを用いてrecordsをfilterメソッドでフィルタリングし、filteredRecordsを更新する。依存配列は[records, searchId, anomalyFilter]。

4.2.2 心電図詳細ページ (/records/[id])

主要機能：波形グラフ表示・AI診断結果表示・PDFエクスポート・医師コメント編集。

| State変数                | 用途・初期値                                                 |
|------------------------|--------------------------------------------------------|
| data: EcgRecord   null | APIから取得した単一ECGレコード。初期値はnull                            |
| chartData: any[]       | Recharts用チャートデータ（{time, voltage}[]）。waveformDataから変換する |
| comment: string        | 医師コメント入力値。data.doctorCommentで初期化する                     |
| loading: boolean       | データ読み込み中フラグ                                            |

波形データ変換処理：waveformDataをJSON.parseしてnumber[]に変換する。配列末尾から連続する0の値をトリミングして心拍部分のみを表示する。Rechartsの{time: index, voltage: value}形式に変換してchartDataに設定する。

PDFエクスポート処理：html2pdf.jsを動的インポートしてreportRef.currentのDOMをA4 PDFに変換する。出力ファイル名はECG\_Report\_{id}.pdfとする。

4.2.3 患者一覧ページ (/patients)

主要機能：患者カード一覧表示・新規患者登録フォーム。

| State変数             | 用途・初期値                                        |
|---------------------|-----------------------------------------------|
| patients: Patient[] | 患者一覧。useEffect内でfetchPatientsを呼び出して設定する       |
| showForm: boolean   | 登録フォーム表示フラグ。初期値はfalse                         |
| newPatient: {...}   | 登録フォームの入力値（name, age, gender）。genderの初期値はMale |

患者登録フロー：handleRegisterがonSubmitで呼ばれる。newPatientのageをintegerに変換してPOST /api/patientsへ送信する。成功後にshowFormをfalseに戻し、newPatientを初期値にリセットし、fetchPatientsでリストを再取得する。

4.3 認証フロー詳細設計

4.3.1 AppLayoutコンポーネント

NextAuthのuseSession()フックでセッション状態を監視する。usePathnameで現在のパスを取得し、以下のロジックで認証ガードを実装する。

- status === "unauthenticated" かつ pathname !== "/" の場合、router.push("/")で強制リダイレクトする
- status === "authenticated" かつ pathname === "/login" などの場合、ダッシュボードへリダイレクトする
- status === "loading" の場合、ローディングスピナーを表示する
- 認証済みの場合のみサイドバーを含むレイアウトを表示する

### 4.3.2 NextAuth CredentialsProvider

authorize関数でユーザー認証を行う。credentials.usernameとcredentials.passwordをバックエンドのPOST /api/auth/loginへ送信する。200レスポンス時はuserオブジェクト（{id, username}）を返す。失敗時はnullを返してエラーを通知する。

## 4.4 コンポーネント型定義

### 4.4.1 EcgRecord型

```
type EcgRecord = {  
  id: number;  
  patient: Patient | null;  
  isAnomaly: boolean;  
  waveformData: string;  
  doctorComment?: string;  
  diagnosisType?: number;  
}
```

### 4.4.2 Patient型

```
type Patient = {  
  id: number;  
  name: string;  
  age: number;  
  gender: string;  
}
```

## 5. データベース詳細設計

### 5.1 DDL定義

#### 5.1.1 usersテーブル

```
CREATE TABLE users (  
  id          INT AUTO_INCREMENT PRIMARY KEY,  
  username    VARCHAR(255) NOT NULL UNIQUE,  
  password    VARCHAR(255) NOT NULL,  
  display_name VARCHAR(255),  
  role        VARCHAR(50)  
);
```

#### 5.1.2 patientsテーブル

```
CREATE TABLE patients (  
  id      INT AUTO_INCREMENT PRIMARY KEY,  
  name    VARCHAR(255) NOT NULL,  
  age     INT,  
  gender  VARCHAR(50)  
);
```

#### 5.1.3 ecg\_recordsテーブル

```
CREATE TABLE ecg_records (  
  id          INT AUTO_INCREMENT PRIMARY KEY,  
  patient_id  INT,  
  recorded_at DATETIME,  
  heart_rate  INT,  
  is_anomaly  TINYINT(1),  
  waveform_data JSON NOT NULL,  
  doctor_comment TEXT,  
  diagnosis_type INT,  
  FOREIGN KEY (patient_id) REFERENCES patients(id)  
    ON DELETE SET NULL ON UPDATE CASCADE  
);
```

## 5.2 インデックス設計

| テーブル        | インデックス対象<br>カラム | 理由                                          |
|-------------|-----------------|---------------------------------------------|
| users       | username        | UNIQUE制約によりunique indexが自動生成される。ログイン時の検索に使用 |
| ecg_records | patient_id      | 外部キーインデックス。患者別レコード検索（searchRecords）に使用      |
| ecg_records | is_anomaly      | 異常フィルタリング検索（searchRecords）の高速化              |
| ecg_records | id（降順）          | findAllByIdOrderByIdDesc()の高速化              |

## 5.3 スキーマ移行ポリシー

spring.jpa.hibernate.ddl-auto=noneを設定し、Hibernateによる自動スキーマ変更を禁止する。スキーマ変更は以下の手順で行う。

- 1. 変更内容のSQLスクリプトを作成し、レビューを受ける
- 2. 外部キー制約が関係する変更の場合、FOREIGN\_KEY\_CHECKS=0で制約を一時的に無効化する
- 3. ALTER TABLE文を実行する
- 4. FOREIGN\_KEY\_CHECKS=1で制約を再有効化する
- 5. SHOW CREATE TABLEで変更内容を確認する

## 6. API入出力仕様詳細

### 6.1 POST /api/auth/login

#### リクエスト仕様

Content-Type: application/json

```
{  "username": "admin",  "password": "password123"}
```

#### レスポンス仕様（成功時）

HTTP 200 OK

```
{  "id": 1,  "username": "admin"}
```

#### レスポンス仕様（エラー時）

| HTTPステータス | ボディ                   | 条件                          |
|-----------|-----------------------|-----------------------------|
| 400       | "missing credentials" | username または password がnull |
| 401       | "user not found"      | 指定usernameのユーザーが存在しない       |
| 401       | "password mismatch"   | パスワードが一致しない                 |

### 6.2 POST /api/ecg/import

#### リクエスト仕様

Content-Type: multipart/form-data

| パラメータ名    | 必須/任意 | 説明                                     |
|-----------|-------|----------------------------------------|
| file      | 必須    | MIT-BIH形式CSVファイル（187列以上）。UTF-8エンコーディング |
| patientId | 任意    | 紐付ける患者のID（整数）。指定しない場合はnullで登録          |

#### レスポンス仕様

| HTTPステータス | ボディ                      |
|-----------|--------------------------|
| 200 OK    | "CSVのインポートとAI解析が完了しました！" |
| 500       | "エラー: {例外メッセージ}"         |



### 6.3 POST /api/predict (AIサービス)

#### リクエスト仕様

Content-Type: application/json

```
{
  "waveform": [0.123, -0.456, 0.789, ...]  // 187個のfloat値
}
```

#### レスポンス仕様 (成功時)

HTTP 200 OK

```
{
  "prediction_code": 2,
  "prediction_name": "VEB (心室性期外収縮)",
  "confidence": "87.23%",
  "is_anomaly": true,
  "generated_report": "心電図解析の結果、心室性期外収縮 (VEB) と判定されました..."
}
```

#### エラー仕様

| HTTPステータス | 条件・detail                                                     |
|-----------|---------------------------------------------------------------|
| 400       | "Waveform data must contain exactly 187 values." - 波形長が187でない |
| 500       | "AI Model is not loaded." - モデルロード失敗                          |
| 500       | 推論処理中の例外メッセージ                                                 |

## 7. エラーハンドリング設計

### 7.1 バックエンドAPIのエラーハンドリング方針

- ・ コントローラ層でtry-catchを使用し、例外発生時にHTTP 500とエラーメッセージを返す
- ・ EcgImportService.processExistingRecordsでは個別レコードのエラーをキャッチしてログ出力し、処理を継続する
- ・ AiInferenceServiceではRestTemplate呼び出しの例外をRuntimeExceptionにラップして上位に伝播させる

### 7.2 AIサービスのエラーハンドリング方針

- ・ FastAPIのHTTPExceptionでエラーを返す
- ・ モデルがロードされていない場合はHTTP 500 (model not loaded) を返す
- ・ 入力値検証エラーはHTTP 400で詳細メッセージを返す
- ・ 推論・レポート生成中の例外は500として返し、サーバーログに詳細を記録する

### 7.3 フロントエンドのエラーハンドリング方針

- ・ fetchメソッドはtry-catchで包み、エラー発生時はconsole.errorでログ出力する
- ・ コメント保存失敗時はalert("保存に失敗しました")でユーザーに通知する
- ・ ローディング状態はloadingフラグで管理し、finallyブロックでsetLoading(false)を確実に実行する

## 8. セキュリティ詳細設計

### 8.1 認証詳細

#### 8.1.1 現行実装

ユーザー名とパスワードをデータベースから取得し、平文比較で認証する。NextAuthのCredentialsProviderが認証フローを管理し、セッションをHttpOnly Cookieで保持する。

#### 8.1.2 改善計画 (bcrypt移行)

パスワード保存時：BCryptPasswordEncoder.encode(rawPassword)でハッシュ化してDBに保存する。

認証時：BCryptPasswordEncoder.matches(rawPassword, encodedPassword)でハッシュと比較する。

### 8.2 CORS設定詳細

allowedOriginPatterns：Vercelデプロイ先URL (https://digital-pulse-psi.vercel.app) とローカル開発URL (http://localhost:3000) を許可する。

allowedMethods：GET, POST, PUT, DELETE, OPTIONSを許可する。

allowedHeaders：Authorization, Content-Typeを許可する。

allowCredentials：trueに設定する (HttpOnly Cookie共有に必要)。

### 8.3 環境変数管理

| 変数名                                   | 対象サービス       | 管理方法                           |
|---------------------------------------|--------------|--------------------------------|
| MYSQLHOST/PORT/USER/PASSWORD/DATABASE | ecg-api      | Renderの環境変数設定画面で管理             |
| OPENAI_API_KEY                        | ecg-ai       | Google Cloud Runのシークレット環境変数で管理 |
| NEXTAUTH_SECRET                       | ecg-frontend | Vercelの環境変数設定画面で管理             |
| NEXT_PUBLIC_API_URL                   | ecg-frontend | Vercelの環境変数設定画面で管理             |

## 9. テスト設計

### 9.1 単体テスト方針

- ・ バックエンドAPI：JUnitとMockitoを使用してService層・Controller層を単体テストする
- ・ AIサービス：pytest（uvicornのTestClient）でエンドポイントの入出力を検証する
- ・ フロントエンド：Jest + React Testing Libraryでコンポーネントのレンダリングとイベントを検証する

### 9.2 主要テストケース

| No.    | テスト対象                     | テスト内容                  | 期待値                                                     |
|--------|---------------------------|------------------------|---------------------------------------------------------|
| UT-001 | POST /api/auth/login      | 正しい認証情報でリクエストする        | 200 OK、{id, username}を返す                                |
| UT-002 | POST /api/auth/login      | 存在しないユーザー名でリクエストする     | 401 Unauthorized (user not found)                       |
| UT-003 | POST /api/auth/login      | パスワード不一致でリクエストする       | 401 Unauthorized (password mismatch)                    |
| UT-004 | POST /api/ecg/import      | 正常なCSVファイルをインポートする     | 200 OK、DBにレコードが保存される                                    |
| UT-005 | GET /api/ecg/search       | patientId=1でフィルタリングする  | patient.id=1のレコードのみ返る                                   |
| UT-006 | POST /api/predict (AI)    | 187点の波形データを送信する        | 200 OK、prediction_code・confidence・generated_reportが含まれる |
| UT-007 | POST /api/predict (AI)    | 186点（不足）の波形データを送信する    | 400 Bad Request                                         |
| UT-008 | RecordDetailPage          | 波形データのゼロトリミングを検証する     | 末尾の連続ゼロが除去され chartDataが更新される                            |
| UT-009 | AppLayout                 | 未認証状態でダッシュボード以外にアクセスする | ルートパス (/) へリダイレクトされる                                    |
| UT-010 | PUT /api/ecg/{id}/comment | 存在するIDへコメントを更新する       | 200 OK、DBのdoctor_commentが更新される                          |

## 10. デプロイ手順

### 10.1 バックエンドAPI (Render)

- 1. GitHubのmainブランチにコードをプッシュする
- 2. Renderのダッシュボードで「Deploy Latest Commit」を実行する
- 3. ビルドログで `./mvnw spring-boot:run` の成功を確認する
- 4. GET `/api/ecg/test` にアクセスして「最新のコードが正常に動いています！」を確認する

### 10.2 AIサービス (Google Cloud Run)

- 1. Dockerイメージをビルドする：`docker build -t ecg-ai .`
- 2. Google Container Registryにプッシュする：`docker push gcr.io/[PROJECT_ID]/ecg-ai`
- 3. Cloud Runにデプロイする：`gcloud run deploy ecg-ai --image gcr.io/[PROJECT_ID]/ecg-ai --region asia-northeast1`
- 4. デプロイ後にGET `/health` で`{"status": "ok", "model_loaded": true}`を確認する

### 10.3 フロントエンド (Vercel)

- 1. GitHubのmainブランチにコードをプッシュする
- 2. VercelがGitHub連携によって自動的にデプロイを実行する
- 3. Vercelダッシュボードでデプロイステータス (Ready) を確認する
- 4. デプロイURLにアクセスしてログイン画面の表示を確認する

## 11. 承認欄

| 役割    | 氏名 | 署名 | 承認日       |
|-------|----|----|-----------|
| 作成者   | 辰田 |    | 2026年5月5日 |
| レビュー者 |    |    |           |
| 承認者   |    |    |           |

以上